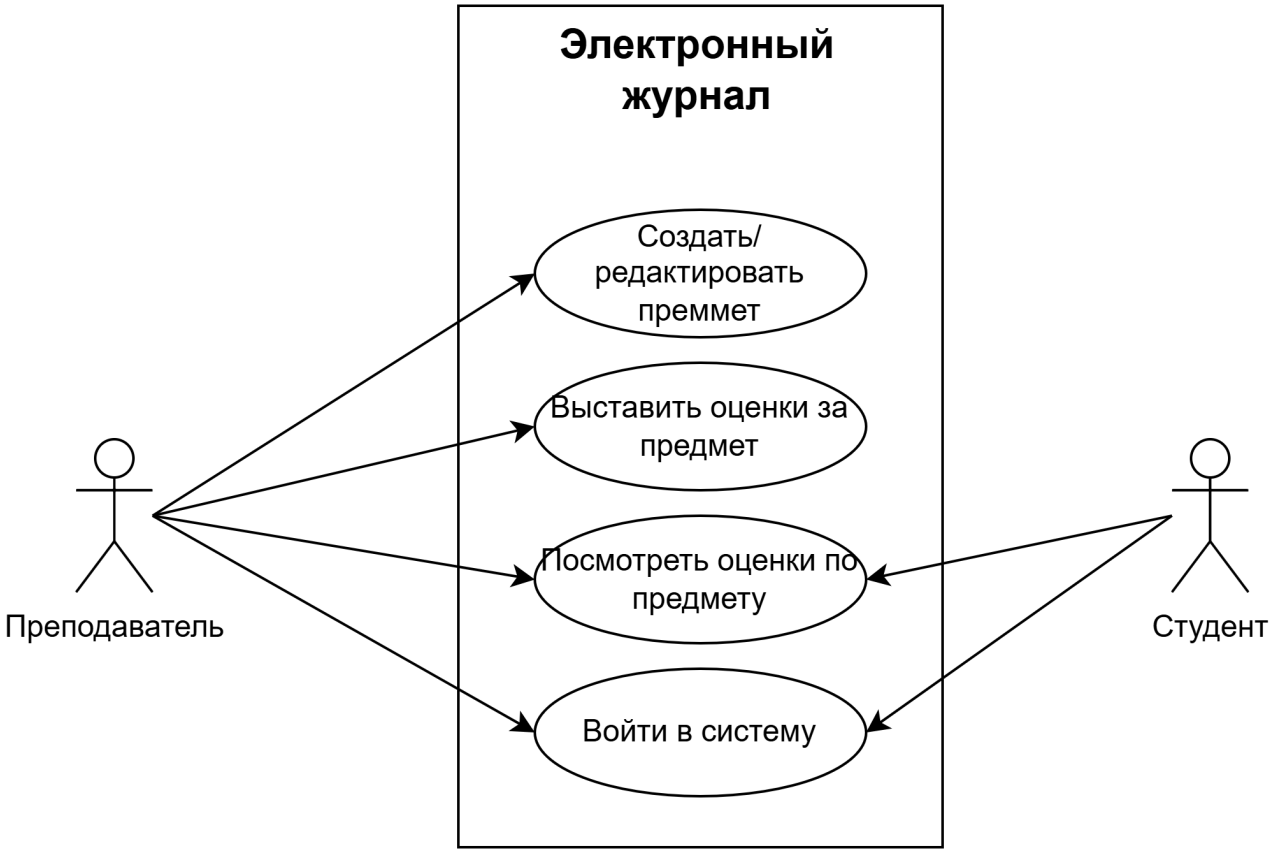
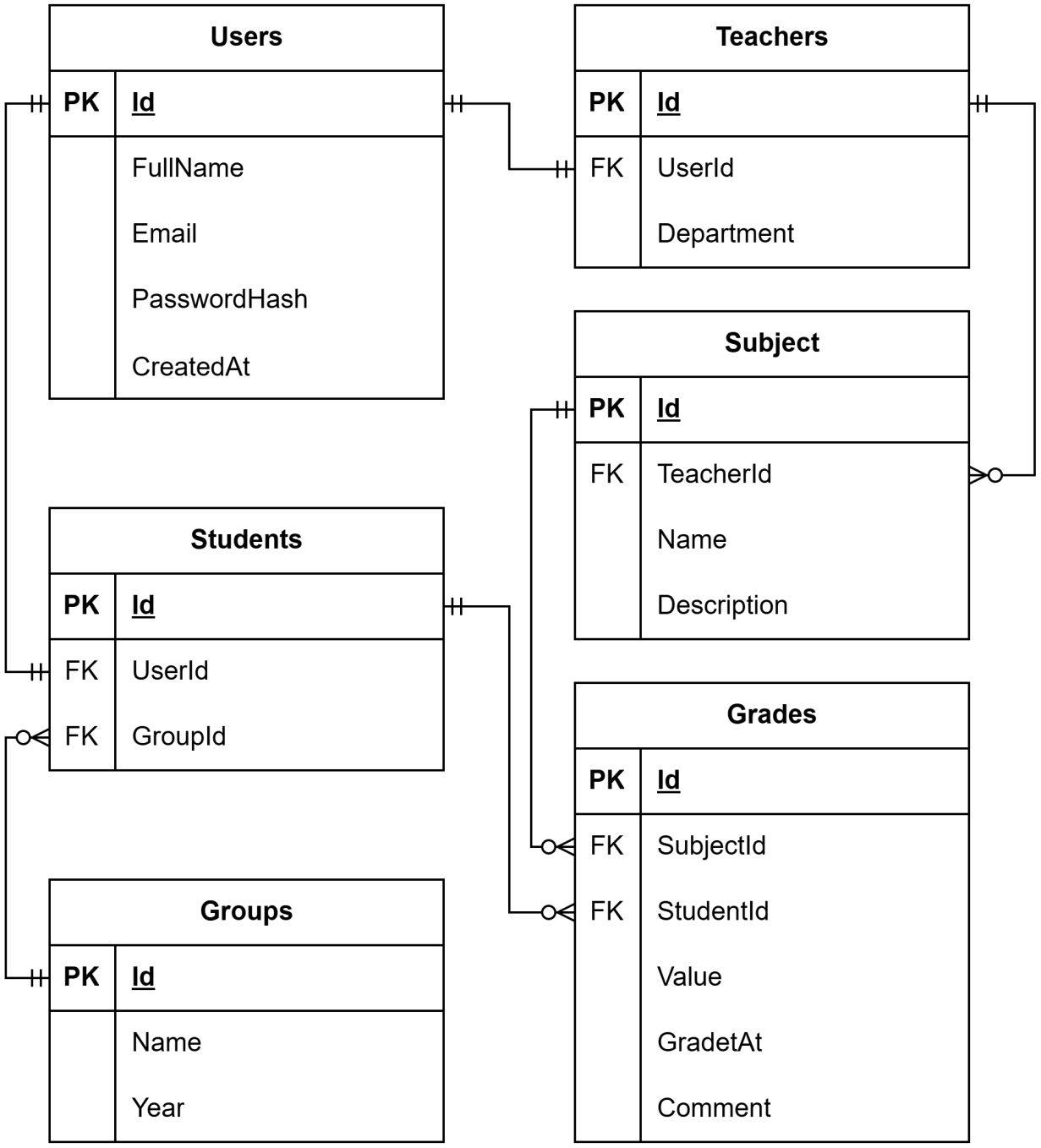
Тема: «Электронный журнал группы»

Use Cases



- Преподаватель входит в систему
- Преподаватель создаёт/редактирует предмет
- Преподаватель выставляет оценку студенту за предмет
- Преподаватель смотрит журнал по своему предмету
- Студент входит в систему
- Студент смотрит свои оценки

ER-диаграмма



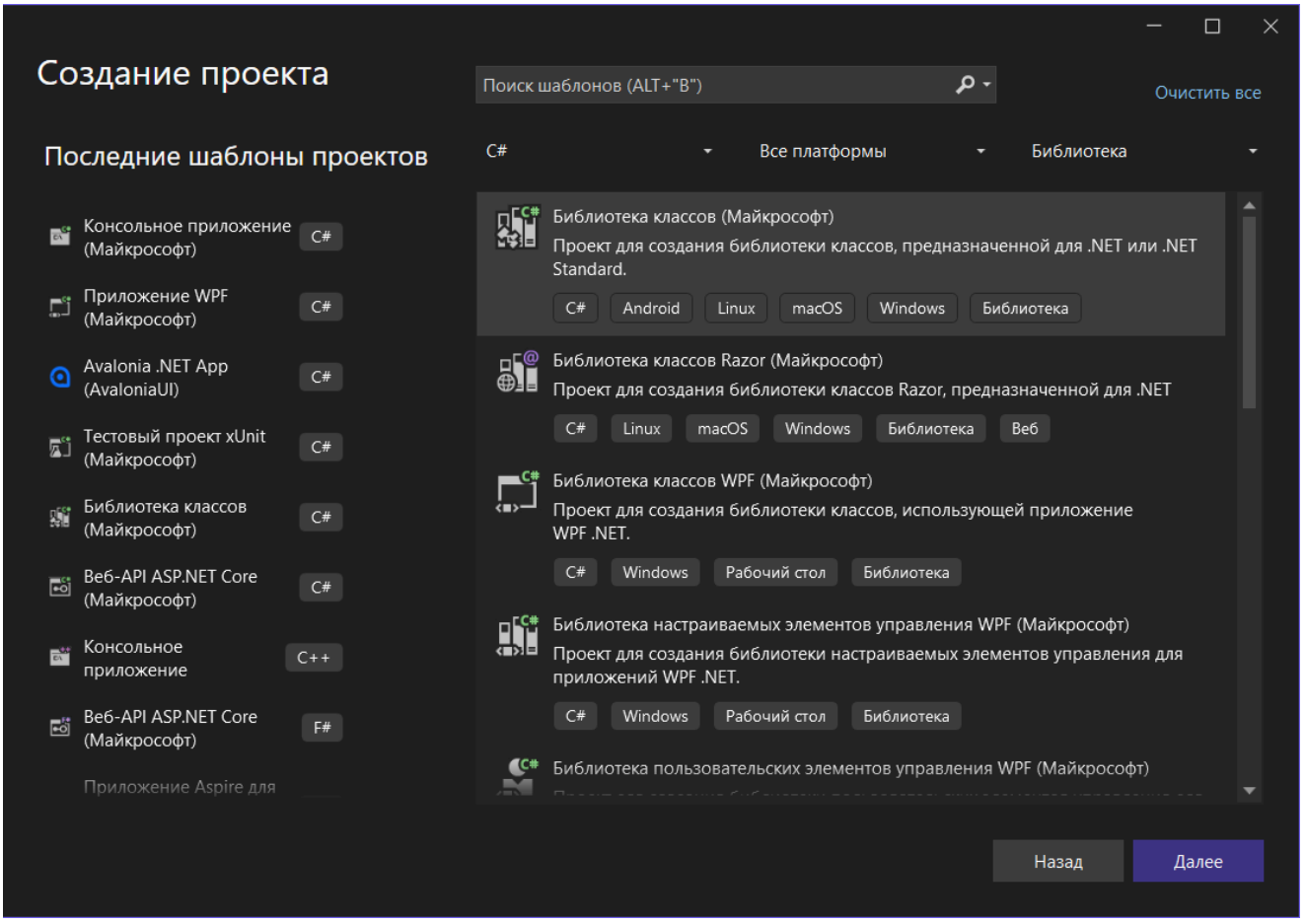
Структура решения

Проект нужно разбить на несколько отдельных частей в зависимости от назначения. Такой подход называется **разделением ответственности**: каждая часть системы знает ровно столько, сколько ей нужно, и не больше. `Models` — общий язык всей системы, к нему имеют доступ все остальные проекты. `Data` работает с базой данных. `Client` общается с API. `Tests` проверяет всё остальное.

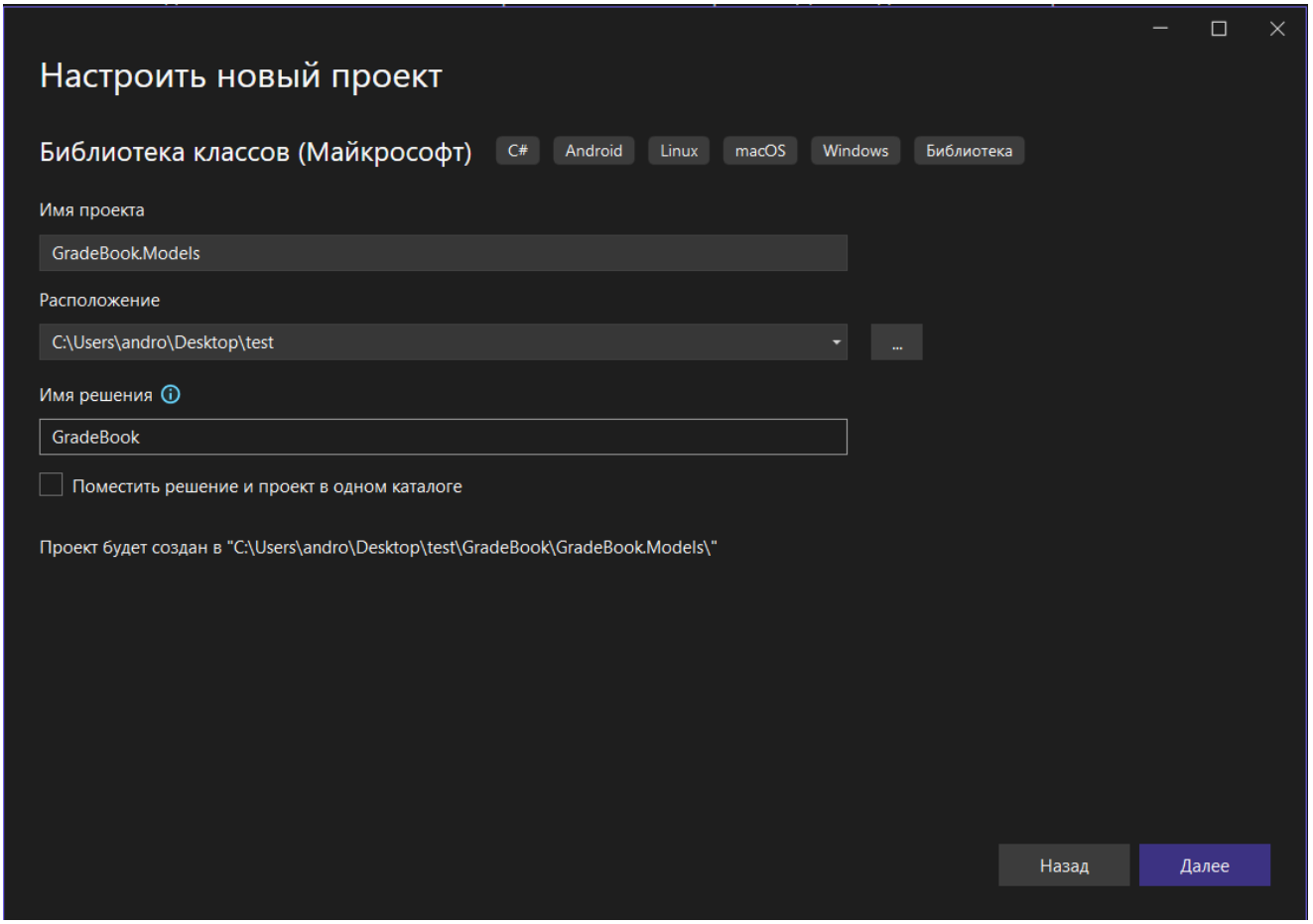
```
GradeBook.sln
├── GradeBook.Models/       ← общий язык системы: модели и DTO
├── GradeBook.Data/         ← работа с БД: EF Core, SQLite, репозитории
├── GradeBook.Client/       ← HTTP-клиент к будущему API
└── GradeBook.Tests/        ← тесты: xUnit + Moq
```

Зависимости между проектами выглядят так: `Data` и `Client` знают о `Models`, потому что работают с теми же классами. `Tests` знает обо всех трёх, потому что тестирует каждый из них. Важно, что `Client` не знает о `Data` — клиент не должен обращаться к базе напрямую, только через API.

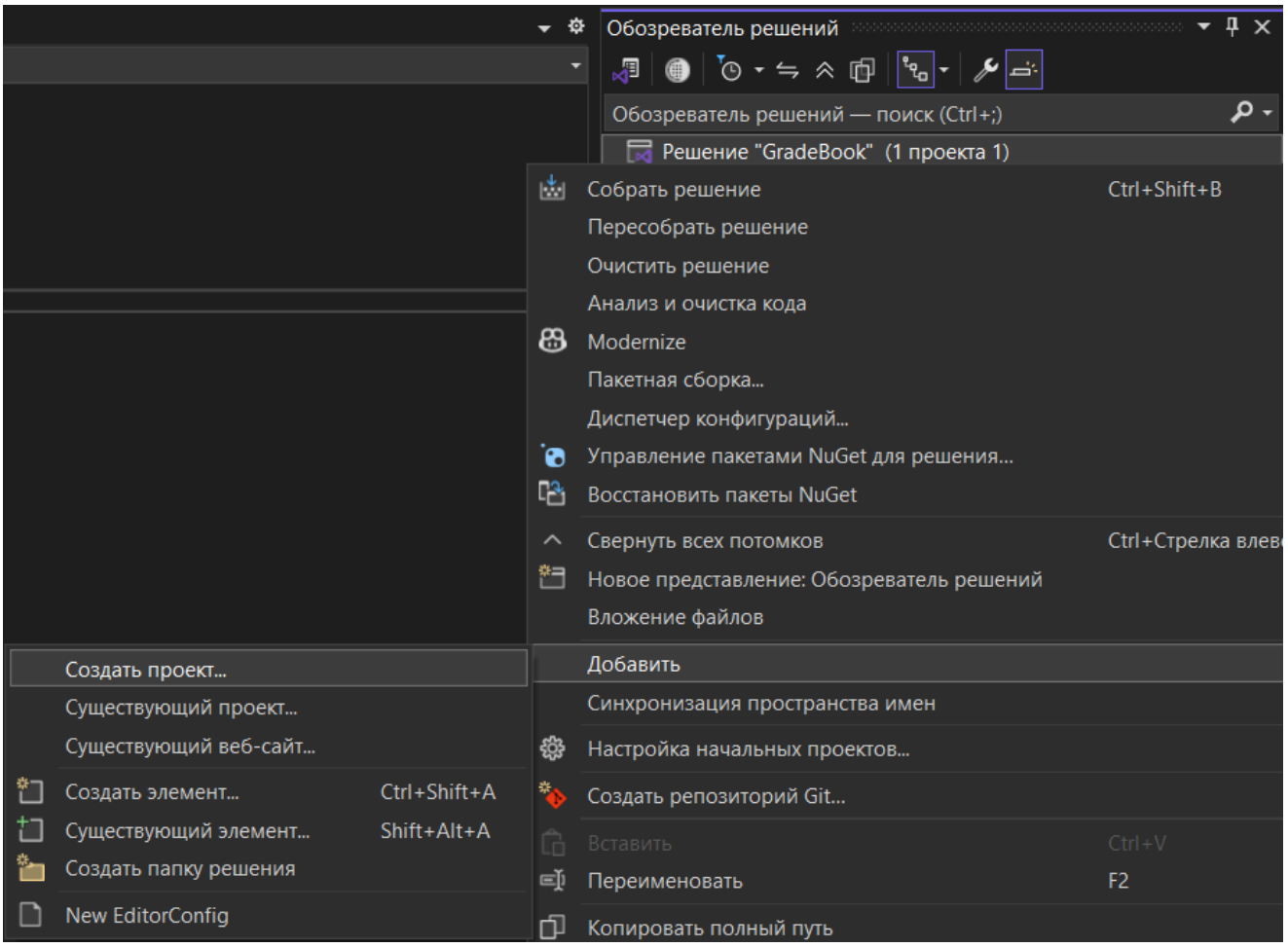
На данном этапе разрабатывается не исполняемый код, а наборы классов, которые будут в дальнейшем использоваться в различных частях проекта. Для создания таких сборок используются проект типа *Библиотека Классов*. Выберете соответствующий шаблон в Visual Studio для создания такого проекта.



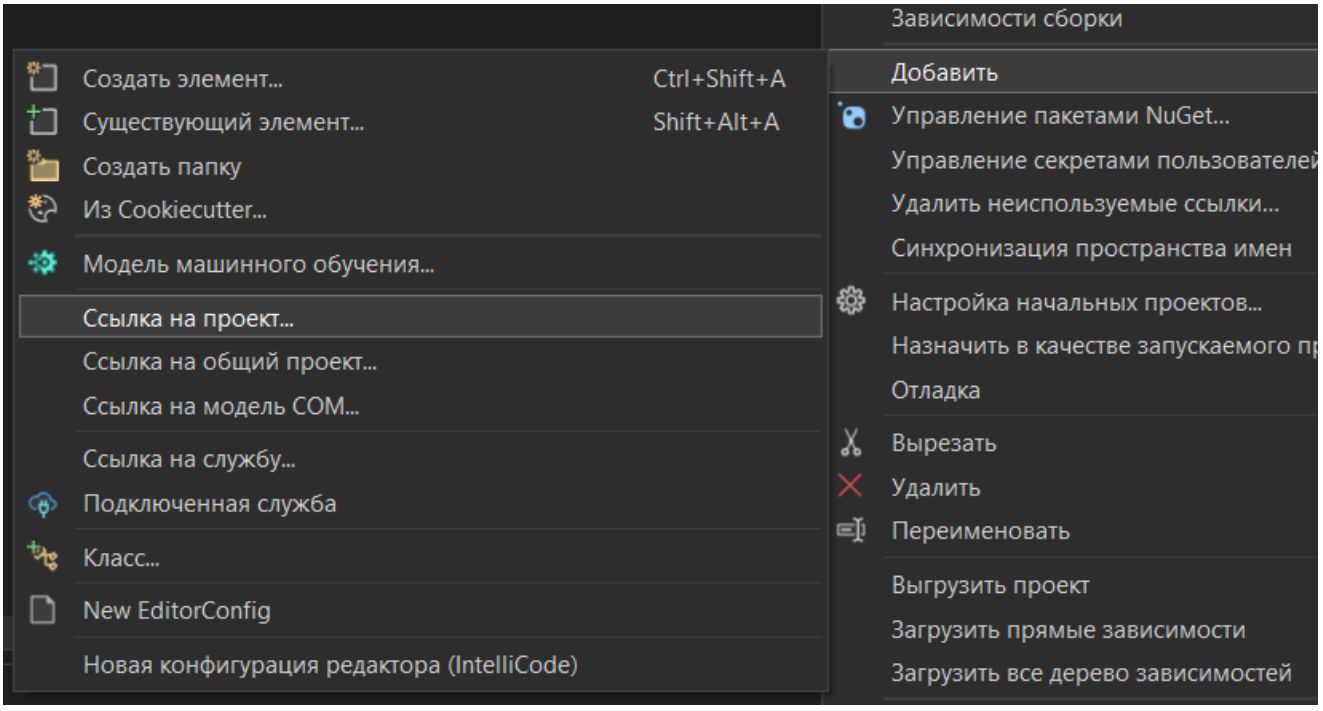
Создаем проект `GradeBook.Models` в решении `GradeBook`.



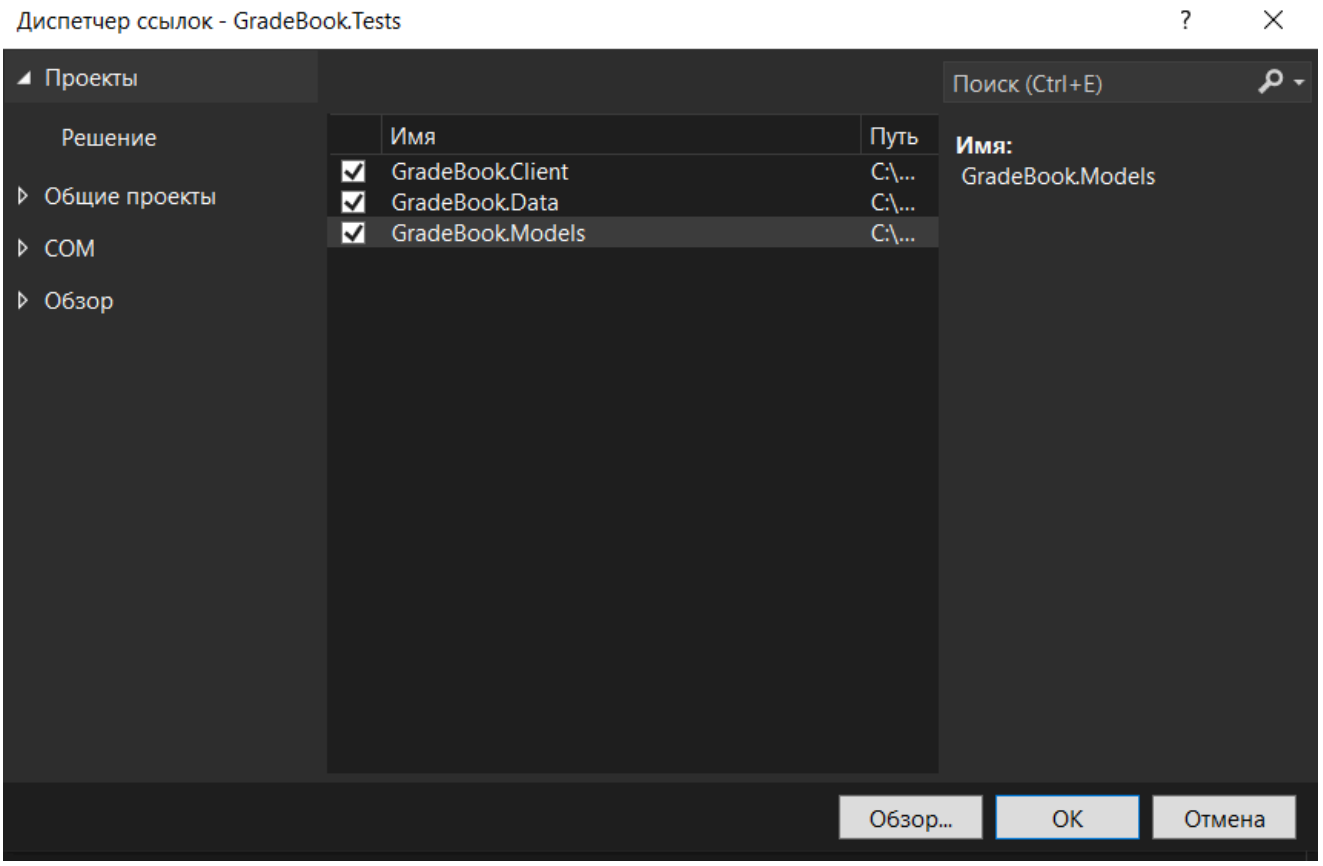
Чтобы создать остальные проекты, в обозревателе решений нажмите ПКМ на текущее решение > *Добавить* > *Создать проект...* . Аналогично создаете остальные проекты с шаблоном *Библиотека классов*. Для тестового проекта выберите шаблон *Тестовый проект xUnit*.



После создания нужно установить зависимости между проектами. Для этого выберите нужный проект в обозревателе решений, нажмите ПКМ > *Добавить* > *Ссылка на проект*.



В открывшемся окне выберите нужные проекты.



То же самое можно сделать через консольные комманды.

```
dotnet new sln -n GradeBook
dotnet new classlib -n GradeBook.Models
dotnet new classlib -n GradeBook.Data
dotnet new classlib -n GradeBook.Client
dotnet new xunit -n GradeBook.Tests

dotnet sln add GradeBook.Models
dotnet sln add GradeBook.Data
dotnet sln add GradeBook.Client
dotnet sln add GradeBook.Tests

dotnet add GradeBook.Data reference GradeBook.Models
dotnet add GradeBook.Client reference GradeBook.Models
dotnet add GradeBook.Tests reference GradeBook.Models
dotnet add GradeBook.Tests reference GradeBook.Data
dotnet add GradeBook.Tests reference GradeBook.Client
```

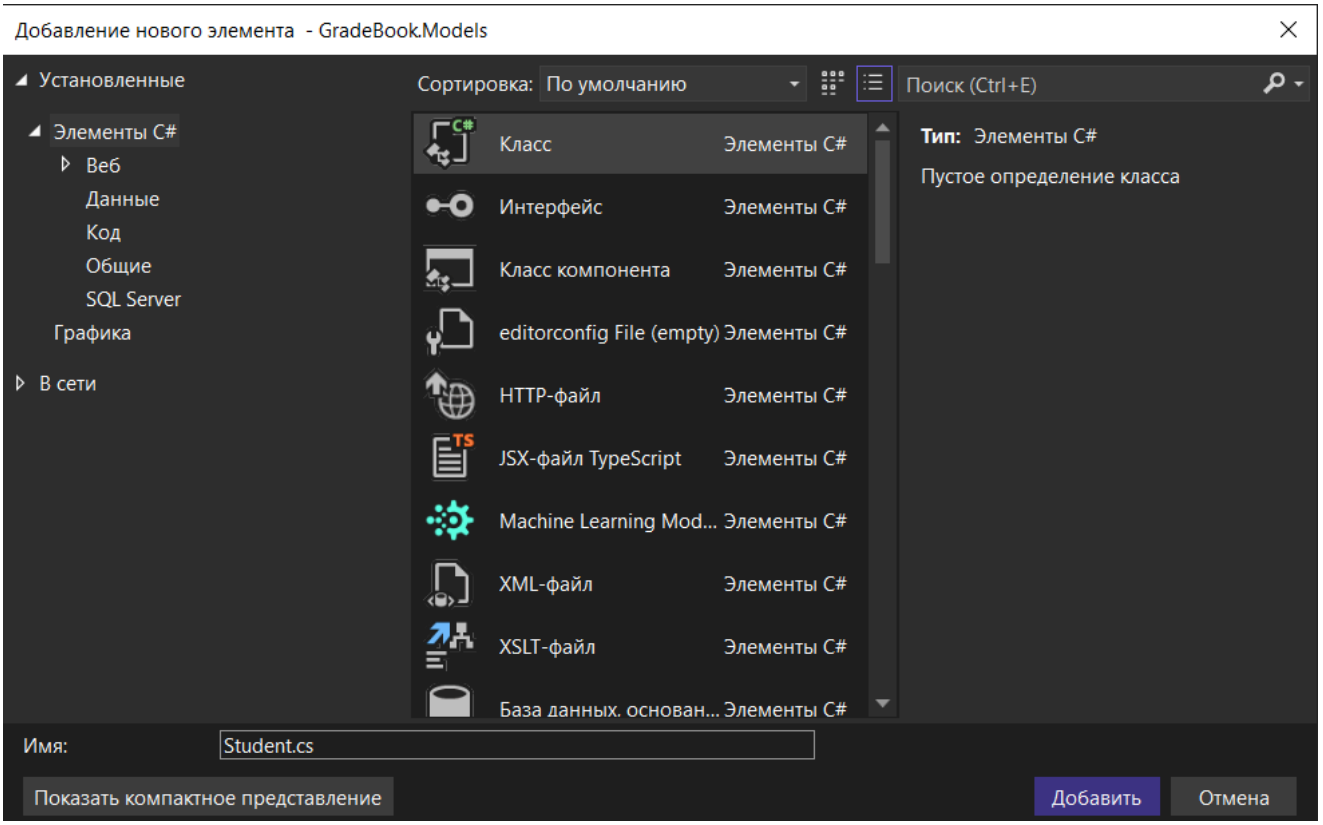
Проект 1: GradeBook.Models

Этот проект — прямое отражение ER-диаграммы в коде. Каждая таблица становится классом, каждый столбец — свойством. Здесь нет никакой логики, только структура данных. Именно поэтому на него ссылаются все остальные проекты: это общий язык всей системы.

Так же здесь находятся **DTO** (Data Transfer Objects) — упрощённые объекты для передачи данных по сети. Они нужны, чтобы не отдавать клиенту лишние поля: например, нет смысла передавать `PasswordHash`, а при создании оценки не нужно передавать весь объект `Grade` целиком — достаточно нескольких полей.

```
GradeBook.Models/
├── Role.cs                 ← перечисление ролей (Teacher, Student)
├── User.cs                 ← базовый пользователь системы
├── Group.cs                ← учебная группа
├── Student.cs              ← студент (связан с User и Group)
├── Teacher.cs              ← преподаватель (связан с User)
├── Subject.cs              ← предмет (принадлежит Teacher)
├── Grade.cs                ← оценка (связывает Subject и Student)
└── DTO/
    ├── GradeCreateDto.cs   ← данные для создания оценки
    ├── SubjectCreateDto.cs ← данные для создания предмета
    └── ...                 ← остальные DTO
```

Чтобы добавит в проект новый .cs-файл или папку, нажмите на него ПКМ в обозревателе решений > *Добавить* > *Создать элемент*, либо *Создать папку*.



Models/Role.cs

Роль пользователя удобно хранить как перечисление — это исключает ошибки со строками вроде `"teahcer"`. Если написать `Role.Teacher`, компилятор проверит корректность за нас.

```
namespace GradeBook.Models;

public enum Role
{
    Teacher,
    Student
}
```

Models/User.cs

`User` — это базовая сущность любого человека в системе. Конкретная роль (студент или преподаватель) описывается отдельными классами `Student` и `Teacher`, которые ссылаются на `User`. Это позволяет хранить специфичные для каждой роли данные отдельно, не раздувая одну таблицу.

```
namespace GradeBook.Models;

public class User
{
    public int Id { get; set; }
    public string FullName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string PasswordHash { get; set; } = string.Empty;
    public Role Role { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}
```

Models/Group.cs

`Group` — это учебная группа. Навигационное свойство `Students` позволит EF Core автоматически подгружать список студентов группы через `.Include()`, без написания JOIN вручную.

```
namespace GradeBook.Models;

public class Group
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;  // например, "A-101"
    public int Year { get; set; }                      // год набора
```

```
    public ICollection<Student> Students { get; set; } = new List<Student>
();
}
```

Models/Student.cs

`Student` связывает пользователя с группой. `UserId` и `GroupId` — это внешние ключи, они же отражают связи из ER-диаграммы. Свойства `User` и `Group` — навигационные: EF Core будет использовать их для JOIN-запросов. Инициализатор `null!` говорит компилятору, что это поле будет заполнено в БД, и не нужно предупреждать о null.

```
namespace GradeBook.Models;

public class Student
{
    public int Id { get; set; }
    public int UserId { get; set; }
    public int GroupId { get; set; }

    public User User { get; set; } = null!;
    public Group Group { get; set; } = null!;
    public ICollection<Grade> Grades { get; set; } = new List<Grade>();
}
```

Models/Teacher.cs

```
namespace GradeBook.Models;

public class Teacher
{
    public int Id { get; set; }
    public int UserId { get; set; }
    public string Department { get; set; } = string.Empty;

    public User User { get; set; } = null!;
    public ICollection<Subject> Subjects { get; set; } = new List<Subject>
();
}
```

Models/Subject.cs

```
namespace GradeBook.Models;

public class Subject
{
    public int Id { get; set; }
    public int TeacherId { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;

    public Teacher Teacher { get; set; } = null!;
    public ICollection<Grade> Grades { get; set; } = new List<Grade>();
}
```

Models/Grade.cs

`Grade` — центральная сущность журнала. Она связывает предмет и студента через внешние ключи `SubjectId` и `StudentId`. Ограничение `Value` от 2 до 5 мы пропишем не здесь, а в `AppDbContext` через Check Constraint — это гарантия на уровне базы данных.

```
namespace GradeBook.Models;

public class Grade
{
    public int Id { get; set; }
    public int SubjectId { get; set; }
    public int StudentId { get; set; }
    public int Value { get; set; }          // от 2 до 5
    public string Comment { get; set; } = string.Empty;
    public DateTime GradedAt { get; set; } = DateTime.UtcNow;

    public Subject Subject { get; set; } = null!;
    public Student Student { get; set; } = null!;
}
```

Models/DTO/GradeCreateDto.cs

DTO нужны, чтобы API принимал только те данные, которые действительно нужны для операции. При создании оценки клиент не должен передавать `GradedAt` или `Id` — сервер проставит их сам.

```
namespace GradeBook.Models.DTO;

public class GradeCreateDto
{
    public int StudentId { get; set; }
    public int SubjectId { get; set; }
    public int Value { get; set; }
    public string Comment { get; set; } = string.Empty;
}
```

Models/DTO/SubjectCreateDto.cs

```
namespace GradeBook.Models.DTO;

public class SubjectCreateDto
{
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
}
```

И т.д.

Проект 2: GradeBook.Data

Этот проект отвечает за всё, что связано с базой данных. Внутри три слоя: `AppDbContext` описывает схему базы и правила связей между таблицами, интерфейсы репозиториев описывают что умеет система, реализации — как именно это делается через EF Core.

Разделение на интерфейс и реализацию принципиально: тесты будут работать с интерфейсами, подставляя вместо реальной базы поддельные объекты через Moq. Контроллеры API тоже будут знать только об интерфейсах — это называется **инверсией зависимостей** и делает систему гибкой и тестируемой.

В этом проекте нужно установить NuGet-пакеты для работы с EF Core и SQLite. Для этого нажмите ПКМ на проект в обозревателе решений и выберите *Управление пакетами NuGet*. После чего установите необходимые пакеты. Обращайте внимание на совместимость версий пакетов и версии .NET в вашем проекте.

То же самое можно сделать через консоль.

```
dotnet add GradeBook.Data package Microsoft.EntityFrameworkCore
dotnet add GradeBook.Data package Microsoft.EntityFrameworkCore.Sqlite
dotnet add GradeBook.Data package Microsoft.EntityFrameworkCore.Design
```

```
GradeBook.Data/
├── AppDbContext.cs              ← главный класс EF Core: таблицы и связи
└── Repositories/
    ├── IRepository.cs           ← общий интерфейс CRUD для любой сущности
    ├── IGradeRepository.cs      ← специфичные методы для оценок
    ├── ISubjectRepository.cs    ← специфичные методы для предметов
    ├── GradeRepository.cs       ← реализация IGradeRepository
    ├── SubjectRepository.cs     ← реализация ISubjectRepository
    └── ...                      ← и т.д. для всех остальных сущностей
```

Data/AppDbContext.cs

`AppDbContext` — это главный класс EF Core. Он описывает, какие таблицы есть в базе, как они называются, какие между ними связи и какие дополнительные ограничения нужно соблюдать. Метод `OnModelCreating` — это место, где мы настраиваем всё то, что нельзя выразить просто названием свойства: уникальные индексы, каскадное удаление, ограничения на значения.

```
using GradeBook.Models;
using Microsoft.EntityFrameworkCore;

namespace GradeBook.Data;

public class AppDbContext : DbContext
{
    public DbSet<User> Users => Set<User>();
    public DbSet<Group> Groups => Set<Group>();
    public DbSet<Student> Students => Set<Student>();
    public DbSet<Teacher> Teachers => Set<Teacher>();
    public DbSet<Subject> Subjects => Set<Subject>();
    public DbSet<Grade> Grades => Set<Grade>();

    public AppDbContext(DbContextOptions<AppDbContext> options) :
base(options) { }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Email уникален — из Use Case "Регистрация"
        modelBuilder.Entity<User>()
            .HasIndex(u => u.Email)
            .IsUnique();

        // Один пользователь — один студент (связь один к одному)
        modelBuilder.Entity<Student>()
            .HasOne(s => s.User)
            .WithOne()
            .HasForeignKey<Student>(s => s.UserId);

        // Один пользователь — один преподаватель
        modelBuilder.Entity<Teacher>()
            .HasOne(t => t.User)
            .WithOne()
            .HasForeignKey<Teacher>(t => t.UserId);

        // Студент принадлежит группе (один ко многим)
        modelBuilder.Entity<Student>()
            .HasOne(s => s.Group)
            .WithMany(g => g.Students)
```

```
            .HasForeignKey(s => s.GroupId);

        // Предмет ведёт один преподаватель
        modelBuilder.Entity<Subject>()
            .HasOne(s => s.Teacher)
            .WithMany(t => t.Subjects)
            .HasForeignKey(s => s.TeacherId);

        // Оценка принадлежит студенту
        modelBuilder.Entity<Grade>()
            .HasOne(g => g.Student)
            .WithMany(s => s.Grades)
            .HasForeignKey(g => g.StudentId);

        // Оценка привязана к предмету
        modelBuilder.Entity<Grade>()
            .HasOne(g => g.Subject)
            .WithMany(s => s.Grades)
            .HasForeignKey(g => g.SubjectId);

        // Оценка может быть только от 2 до 5 — ограничение на уровне БД
        modelBuilder.Entity<Grade>()
            .ToTable(t => t.HasCheckConstraint("CK_Grade_Value", "Value >= 2
AND Value <= 5"));
    }
}
```

Интерфейсы репозиториев

Data/Repositories/IRepository.cs

`IRepository<T>` — это общий контракт для любой сущности. Он содержит стандартные CRUD-операции, которые одинаковы для всех. Используем обобщения (generics), чтобы не писать один и тот же код для каждого класса отдельно.

```
namespace GradeBook.Data.Repositories;

public interface IRepository<T> where T : class
{
    Task<T?> GetByIdAsync(int id);
    Task<IEnumerable<T>> GetAllAsync();
    Task AddAsync(T entity);
    Task UpdateAsync(T entity);
    Task DeleteAsync(int id);
}
```

Data/Repositories/IGradeRepository.cs

Специфичные интерфейсы расширяют базовый и добавляют методы, которые отражают конкретные Use Cases. Важно называть методы так, чтобы было понятно, какой сценарий они реализуют.

```
using GradeBook.Models;

namespace GradeBook.Data.Repositories;

public interface IGradeRepository : IRepository<Grade>
{
    // Use Case: "Преподаватель смотрит журнал по предмету"
    Task<IEnumerable<Grade>> GetBySubjectAsync(int subjectId);

    // Use Case: "Студент смотрит свои оценки"
    Task<IEnumerable<Grade>> GetByStudentAsync(int studentId);
}
```

Data/Repositories/ISubjectRepository.cs

```
using GradeBook.Models;

namespace GradeBook.Data.Repositories;

public interface ISubjectRepository : IRepository<Subject>
{
    // Use Case: "Преподаватель смотрит свои предметы"
    Task<IEnumerable<Subject>> GetByTeacherAsync(int teacherId);
}
```

И т.д.

Реализации репозиториев

Реализация содержит конкретные запросы к базе через EF Core. Обратите внимание на `.Include()` — это способ подгрузить связанные данные одним запросом. Без него свойства-навигации (`Student.User` , `Grade.Subject`) будут `null` .

Data/Repositories/GradeRepository.cs

```
using GradeBook.Models;
using Microsoft.EntityFrameworkCore;

namespace GradeBook.Data.Repositories;

public class GradeRepository : IGradeRepository
{
    private readonly AppDbContext _context;

    public GradeRepository(AppDbContext context)
    {
        _context = context;
    }

    public async Task<Grade?> GetByIdAsync(int id) =>
        await _context.Grades
            .Include(g => g.Student).ThenInclude(s => s.User)
            .Include(g => g.Subject)
            .FirstOrDefaultAsync(g => g.Id == id);

    public async Task<IEnumerable<Grade>> GetAllAsync() =>
        await _context.Grades.ToListAsync();

    // Для журнала преподавателя: все оценки по предмету,
    // отсортированные по имени студента для удобства чтения
    public async Task<IEnumerable<Grade>> GetBySubjectAsync(int subjectId)
=>
        await _context.Grades
            .Where(g => g.SubjectId == subjectId)
            .Include(g => g.Student).ThenInclude(s => s.User)
            .OrderBy(g => g.Student.User.FullName)
            .ToListAsync();

    // Для личного кабинета студента: его оценки,
    // последние сначала
    public async Task<IEnumerable<Grade>> GetByStudentAsync(int studentId)
=>
        await _context.Grades
            .Where(g => g.StudentId == studentId)
            .Include(g => g.Subject)
```

```
            .OrderByDescending(g => g.GradedAt)
            .ToListAsync();

    public async Task AddAsync(Grade grade)
    {
        await _context.Grades.AddAsync(grade);
        await _context.SaveChangesAsync();
    }

    public async Task UpdateAsync(Grade grade)
    {
        _context.Grades.Update(grade);
        await _context.SaveChangesAsync();
    }

    public async Task DeleteAsync(int id)
    {
        var grade = await GetByIdAsync(id);
        if (grade is not null)
        {
            _context.Grades.Remove(grade);
            await _context.SaveChangesAsync();
        }
    }
}
```

Data/Repositories/SubjectRepository.cs

```
using GradeBook.Models;
using Microsoft.EntityFrameworkCore;

namespace GradeBook.Data.Repositories;

public class SubjectRepository : ISubjectRepository
{
    private readonly AppDbContext _context;

    public SubjectRepository(AppDbContext context)
    {
        _context = context;
    }

    public async Task<Subject?> GetByIdAsync(int id) =>
        await _context.Subjects
            .Include(s => s.Teacher).ThenInclude(t => t.User)
            .FirstOrDefaultAsync(s => s.Id == id);

    public async Task<IEnumerable<Subject>> GetAllAsync() =>
        await _context.Subjects.ToListAsync();
```

```
    // Список предметов конкретного преподавателя
    public async Task<IEnumerable<Subject>> GetByTeacherAsync(int teacherId)
=>
        await _context.Subjects
            .Where(s => s.TeacherId == teacherId)
            .ToListAsync();

    public async Task AddAsync(Subject subject)
    {
        await _context.Subjects.AddAsync(subject);
        await _context.SaveChangesAsync();
    }

    public async Task UpdateAsync(Subject subject)
    {
        _context.Subjects.Update(subject);
        await _context.SaveChangesAsync();
    }

    public async Task DeleteAsync(int id)
    {
        var subject = await GetByIdAsync(id);
        if (subject is not null)
        {
            _context.Subjects.Remove(subject);
            await _context.SaveChangesAsync();
        }
    }
}
```

И т.д.

Проект 3: GradeBook.Client

Этот проект — HTTP-клиент к будущему API. API ещё не написан, но мы уже знаем его структуру из Use Cases, поэтому можем написать клиент заранее. Когда студенты пройдут ASP.NET и напишут сервер, клиент уже будет готов и просто начнёт работать.

Каждый метод клиента — это один HTTP-запрос, соответствующий одному Use Case. `HttpClient` — стандартный класс .NET для выполнения HTTP-запросов. `System.Net.Http.Json` добавляет удобные методы `GetFromJsonAsync` и `PostAsJsonAsync`, которые автоматически сериализуют и десериализуют JSON.

Установите в этом проекте нужные NuGet-пакеты, через интерфейс Visual Studio, либо через консоль.

```
dotnet add GradeBook.Client package System.Net.Http.Json
```

```
GradeBook.Client/
├── ApiClientBase.cs    ← базовый класс с настройкой HttpClient и токена
├── AuthClient.cs       ← регистрация и вход (Use Cases авторизации)
├── GradesClient.cs     ← работа с оценками (Use Cases оценок)
└── SubjectsClient.cs   ← работа с предметами (Use Cases предметов)
```

Client/ApiClientBase.cs

`ApiClientBase` содержит общую логику: хранит `HttpClient` и умеет устанавливать JWT-токен (специальный код для аутентификации, более подробно изучим это на следующих лекциях) после входа в систему. Все остальные клиенты наследуются от него и получают эту функциональность.

```
using System.Net.Http.Headers;

namespace GradeBook.Client;

public abstract class ApiClientBase
{
    protected readonly HttpClient _http;

    protected ApiClientBase(HttpClient http)
    {
        _http = http;
    }

    // Вызывается после успешного логина — токен будет
    // автоматически добавляться ко всем последующим запросам
    public void SetAuthToken(string token)
    {
```

```
        _http.DefaultRequestHeaders.Authorization =
            new AuthenticationHeaderValue("Bearer", token);
    }
}
```

Client/AuthClient.cs

```
using System.Net.Http.Json;

namespace GradeBook.Client;

public class AuthClient : ApiClientBase
{
    public AuthClient(HttpClient http) : base(http) { }

    // Use Case: "Зарегистрироваться"
    public async Task<string?> RegisterAsync(string fullName, string email,
string password)
    {
        var response = await _http.PostAsJsonAsync("api/auth/register", new
        {
            fullName,
            email,
            password
        });
        response.EnsureSuccessStatusCode();
        var result = await response.Content.ReadFromJsonAsync<TokenResponse>
();
        return result?.Token;
    }

    // Use Case: "Войти в систему"
    public async Task<string?> LoginAsync(string email, string password)
    {
        var response = await _http.PostAsJsonAsync("api/auth/login", new
        {
            email,
            password
        });
        response.EnsureSuccessStatusCode();
        var result = await response.Content.ReadFromJsonAsync<TokenResponse>
();
        return result?.Token;
    }
}

// Вспомогательный класс для десериализации ответа сервера с токеном
internal class TokenResponse
{
```

```
    public string Token { get; set; } = string.Empty;
}
```

Client/GradesClient.cs

```
using System.Net.Http.Json;
using GradeBook.Models;
using GradeBook.Models.DTO;

namespace GradeBook.Client;

public class GradesClient : ApiClientBase
{
    public GradesClient(HttpClient http) : base(http) { }

    // Use Case: "Преподаватель смотрит журнал по предмету"
    public async Task<List<Grade>?> GetBySubjectAsync(int subjectId) =>
        await _http.GetFromJsonAsync<List<Grade>>
($"api/grades/subject/{subjectId}");

    // Use Case: "Студент смотрит свои оценки"
    public async Task<List<Grade>?> GetMyGradesAsync(int studentId) =>
        await _http.GetFromJsonAsync<List<Grade>>
($"api/grades/student/{studentId}");

    // Use Case: "Преподаватель выставляет оценку"
    // EnsureSuccessStatusCode бросит исключение если сервер вернул 4xx/5xx
    public async Task<Grade?> CreateGradeAsync(GradeCreateDto dto)
    {
        var response = await _http.PostAsJsonAsync("api/grades", dto);
        response.EnsureSuccessStatusCode();
        return await response.Content.ReadFromJsonAsync<Grade>();
    }

    // Use Case: "Преподаватель исправляет оценку"
    public async Task UpdateGradeAsync(int id, GradeCreateDto dto)
    {
        var response = await _http.PutAsJsonAsync($"api/grades/{id}", dto);
        response.EnsureSuccessStatusCode();
    }

    // Use Case: "Преподаватель удаляет оценку"
    public async Task DeleteGradeAsync(int id)
    {
        var response = await _http.DeleteAsync($"api/grades/{id}");
        response.EnsureSuccessStatusCode();
    }
}
```

Client/SubjectsClient.cs

```
using System.Net.Http.Json;
using GradeBook.Models;
using GradeBook.Models.DTO;

namespace GradeBook.Client;

public class SubjectsClient : ApiClientBase
{
    public SubjectsClient(HttpClient http) : base(http) { }

    // Use Case: "Преподаватель смотрит свои предметы"
    public async Task<List<Subject>?> GetByTeacherAsync(int teacherId) =>
        await _http.GetFromJsonAsync<List<Subject>>
($"api/subjects/teacher/{teacherId}");

    // Use Case: "Преподаватель создаёт предмет"
    public async Task<Subject?> CreateSubjectAsync(SubjectCreateDto dto)
    {
        var response = await _http.PostAsJsonAsync("api/subjects", dto);
        response.EnsureSuccessStatusCode();
        return await response.Content.ReadFromJsonAsync<Subject>();
    }

    // Use Case: "Преподаватель удаляет предмет"
    public async Task DeleteSubjectAsync(int id)
    {
        var response = await _http.DeleteAsync($"api/subjects/{id}");
        response.EnsureSuccessStatusCode();
    }
}
```

Проект 4: GradeBook.Tests

Тесты — это код, который проверяет, что остальной код работает правильно. Здесь два вида тестов. Первый — тесты репозиториев через базу данных в памяти (InMemory): реальный код, реальный EF Core, но вместо файла SQLite — временная база, которая существует только во время теста. Второй — тесты HTTP-клиента через `Mock`: вместо реального сервера подставляется объект, который притворяется сервером и возвращает заранее заданные ответы.

Каждый тест следует структуре **Arrange — Act — Assert**: сначала готовим данные, потом выполняем действие, потом проверяем результат.

Установите в этом проекте нужные NuGet-пакеты, через интерфейс Visual Studio, либо через консоль.

```
dotnet add GradeBook.Tests package Moq
dotnet add GradeBook.Tests package Microsoft.EntityFrameworkCore.InMemory
```

```
GradeBook.Tests/
├── GradeRepositoryTests.cs   ← тесты репозитория через InMemory БД
├── GradesClientTests.cs      ← тесты HTTP-клиента через Mock
└── ...
```

Tests/GradeRepositoryTests.cs

```
using GradeBook.Data;
using GradeBook.Data.Repositories;
using GradeBook.Models;
using Microsoft.EntityFrameworkCore;
using Xunit;

namespace GradeBook.Tests;

public class GradeRepositoryTests
{
    // Каждый тест получает свежую пустую базу в памяти.
    // Guid.NewGuid() гарантирует уникальное имя — тесты не мешают друг
другу,
    // даже если запускаются параллельно
    private AppDbContext CreateContext()
    {
        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .Options;
        return new AppDbContext(options);
    }
```

```
    [Fact]
    public async Task
GetBySubjectAsync_ShouldReturnOnlyGradesForThatSubject()
    {
        // Arrange: добавляем оценки по двум разным предметам
        await using var context = CreateContext();
        var repo = new GradeRepository(context);

        await context.Grades.AddRangeAsync(
            new Grade { SubjectId = 1, StudentId = 1, Value = 5 },
            new Grade { SubjectId = 1, StudentId = 2, Value = 4 },
            new Grade { SubjectId = 2, StudentId = 1, Value = 3 }  // другой
предмет
        );
        await context.SaveChangesAsync();

        // Act: запрашиваем только первый предмет
        var result = await repo.GetBySubjectAsync(subjectId: 1);

        // Assert: должны вернуться только две оценки по первому предмету
        Assert.Equal(2, result.Count());
        Assert.All(result, g => Assert.Equal(1, g.SubjectId));
    }

    [Fact]
    public async Task GetByStudentAsync_ShouldReturnOnlyThatStudentsGrades()
    {
        // Arrange: добавляем оценки двух разных студентов
        await using var context = CreateContext();
        var repo = new GradeRepository(context);

        await context.Grades.AddRangeAsync(
            new Grade { SubjectId = 1, StudentId = 7, Value = 4 },
            new Grade { SubjectId = 2, StudentId = 7, Value = 5 },
            new Grade { SubjectId = 1, StudentId = 9, Value = 3 }  // другой
студент
        );
        await context.SaveChangesAsync();

        // Act
        var result = await repo.GetByStudentAsync(studentId: 7);

        // Assert: только оценки студента 7
        Assert.Equal(2, result.Count());
        Assert.All(result, g => Assert.Equal(7, g.StudentId));
    }
}
```

Tests/GradesClientTests.cs

```
using System.Net;
using System.Text.Json;
using GradeBook.Client;
using GradeBook.Models;
using GradeBook.Models.DTO;
using Moq;
using Moq.Protected;
using Xunit;

namespace GradeBook.Tests;

public class GradesClientTests
{
    // Создаём HttpClient с подменённым транспортом.
    // Вместо реального сетевого соединения — Mock, который
    // возвращает заранее заданный ответ. Сервер запускать не нужно.
    private HttpClient CreateMockHttpClient(HttpResponseMessage response)
    {
        var handlerMock = new Mock<HttpMessageHandler>();
        handlerMock
            .Protected()
            .Setup<Task<HttpResponseMessage>>(
                "SendAsync",
                ItExpr.IsAny<HttpRequestMessage>(),
                ItExpr.IsAny<CancellationToken>())
            .ReturnsAsync(response);

        return new HttpClient(handlerMock.Object)
        {
            BaseAddress = new Uri("https://localhost:7000/")
        };
    }

    [Fact]
    public async Task
CreateGradeAsync_ShouldReturnCreatedGrade_WhenApiReturnsOk()
    {
        // Arrange: готовим фейковый ответ сервера с оценкой
        var expected = new Grade { Id = 1, StudentId = 3, SubjectId = 2,
Value = 5 };
        var json = JsonSerializer.Serialize(expected);
        var fakeResponse = new HttpResponseMessage(HttpStatusCode.OK)
        {
            Content = new StringContent(json, System.Text.Encoding.UTF8,
"application/json")
        };
```

```
        var client = new GradesClient(CreateMockHttpClient(fakeResponse));
        var dto = new GradeCreateDto { StudentId = 3, SubjectId = 2, Value =
5 };

        // Act
        var result = await client.CreateGradeAsync(dto);

        // Assert: клиент правильно десериализовал ответ
        Assert.NotNull(result);
        Assert.Equal(5, result.Value);
        Assert.Equal(3, result.StudentId);
    }

    [Fact]
    public async Task
DeleteGradeAsync_ShouldNotThrow_WhenApiReturnsNoContent()
    {
        // Arrange: сервер возвращает 204 No Content — стандартный ответ на
DELETE
        var fakeResponse = new
HttpResponseMessage(HttpStatusCode.NoContent);
        var client = new GradesClient(CreateMockHttpClient(fakeResponse));

        // Act & Assert: метод не должен бросать исключение
        var exception = await Record.ExceptionAsync(() =>
client.DeleteGradeAsync(1));
        Assert.Null(exception);
    }
}
```

И т.д.

Итог: связь между Use Cases и кодом

Каждый элемент системы имеет чёткое происхождение из Use Cases и ER-диаграммы:

```
Use Case                                → Код
──────────────────────────────────────────────────────────────────────
"Студент / Преподаватель входит"        → AuthClient.LoginAsync()
"Преподаватель создаёт предмет"         → SubjectsClient.CreateSubjectAsync()
                                          SubjectRepository.AddAsync()
"Преподаватель выставляет оценку"       → GradesClient.CreateGradeAsync()
                                          GradeRepository.AddAsync()
"Преподаватель смотрит журнал"          → GradesClient.GetBySubjectAsync()
                                          GradeRepository.GetBySubjectAsync()
"Студент смотрит свои оценки"           →
GradesClient.GetByStudentGradesAsync()
                                          GradeRepository.GetByStudentAsync()

ER-сущность                             → Код
──────────────────────────────────────────────────────────────────────
Users                                   → User.cs + Role.cs
Teachers                                → Teacher.cs
Students                                → Student.cs
Groups                                  → Group.cs
Subjects                                → Subject.cs
Grades                                  → Grade.cs
```

На следующих этапа, когда вы пройдете WPF, нужно будет добавить проект `GradeBook.Desktop`, и связать с `GradeBook.Models` и `GradeBook.Client`, а после того как вы пройдете ASP.NET, нужно будет создать проект `GradeBook.Api`, подключат к нему `GradeBook.Data` и добавят контроллеры — по одному на каждую группу Use Cases. Каркас к тому моменту должен быть уже готов и его можно будет сразу использовать.